

ST2195 Programming for Data Science

Random Walk &
Flight Analysis from 2000 -2004

Angel Fortuna Maharani

Student ID : 230667226

Contents

1.0 Metropolis Random Walk	3
2.0 Flights Analysis 2000 - 2004	5
2.1.1 Problem Analysis.....	5
2.1.2 Data Collection	5
2.1.3 Data Cleaning, Preprocessing, and Wrangling.....	5
2.1.4 Visualization	5
2.2.1 Problem Analysis.....	7
2.2.2 Data Cleaning, Preprocessing, and Wrangling	7
2.2.3 Visualisation	7
3.1.1 Problem Analysis.....	8
3.1.2 Data Cleaning, Preprocessing, and Wrangling	8
3.1.3 Visualisation	9

1.0 Metropolis Random Walk

Metropolis-Hasting's algorithm is using Markov Chain Monte Carlo method where random samples will be generated and each sample will depend on the previous sample. In this coursework, the random numbers will be generated based on $f(x) = \frac{1}{2} e^{(-|x|)}$ which is Laplace Distribution. These samples are not directly generated from the distribution. Instead, it is taken randomly through normal distribution with mean X_{i-1} . Then, these samples will be processed through rules of ratio and u (random samples from uniform distribution). As more samples are generated, it will look like the actual distribution ($f(x)$). The random number which denoted as X^* have mean x_{i-1} that depends on the previous sample. X_i value will depend on X_{i-1} but not on X_{i-2} as it follows the memoryless property of Markov.

At first, the function of $f(x)$ will be created. Then, a new function will be generated to generates the random number. It will contain parameters of initial value X_0 , num_of_sample ($N=10000$), and s which is the step size that equals to 1. The function call "step" in Rmd. It will generate N random number, with mean X_{i-1} and standard deviation s . After that, it will use the function $f(x)$ on the random number generated. It will also store the previous number since it will be useful for ratio (r) calculation. Then " u " will contain the random number from uniform distribution, which will being used in equivalent criterion which the logarithm rules. It will take logarithm of u and compare with the logarithm of ratio r calculated before. If log of u less than log of r , then X^* will be stored as random number. Otherwise, it will be rejected and previous number will be stored again into the random number. These steps will be iterated until 10000 times based on the question provided.

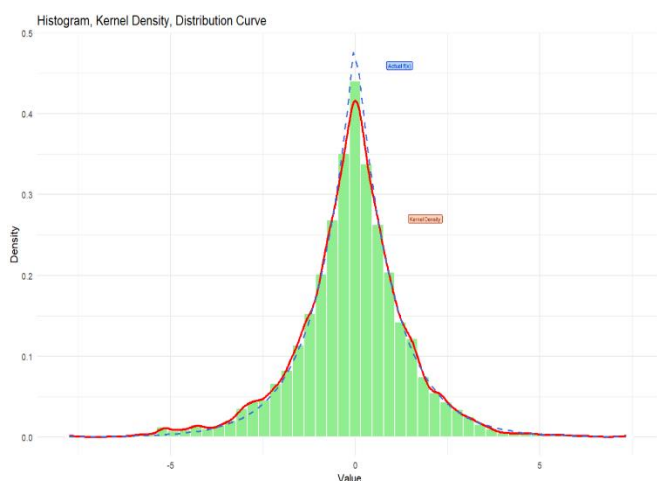


Figure 1.1 Histogram, Kernel Density, Distribution Curve in R

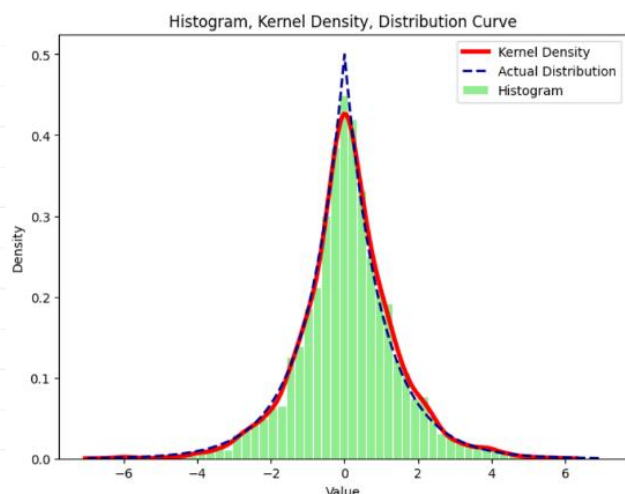


Figure 1.2 Histogram, Kernel Density, Distribution Curve in Python

The graph visualizes the distribution of 10000 random numbers that have been stored before, with the standard deviation 1. It's useful to see whether the Metropolis Hastings random number generated before are able to converge into actual distribution or not. Here, the distribution here looks like leptokurtic, where the tails are fatter. The kernel density is the red curve, which shows the overall shapes of histogram in a smooth curve. This is the estimated curve from the samples. While the blue colour curve shows the actual curve by using the function formula ($f(x)$). This is the target distribution that is being aimed. It can be compared between actual and estimated samples, between red and blue colour. The estimated samples are close and a good approximation towards actual distribution. It's a good indicator of algorithm that successfully converge to the actual distribution.

In order to answer part 1b, a function will be generated. It will be created with parameter N which is 10000, s (step size) that is 1, and J that equals to 4. At first, the random number will be generated so that the initial value X_0 will be different in each chain and it will use the function “step” that have been created in part a. It will then calculate the mean and variance, which will be useful to calculate between sample variance and $\hat{R}$. The function will return $\hat{R}$ value. The function will be used to vary the s from 0.001 to 1 by 0.005. So, it will be around 200 $\hat{R}$ values and stored in data frame that will be plot in graph below.

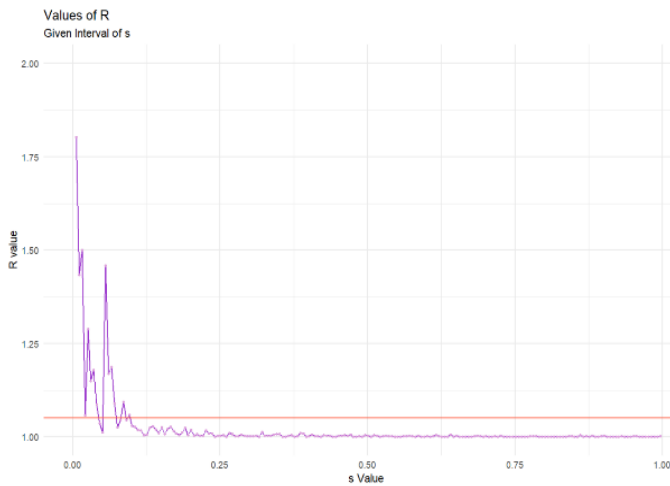


Figure 1.3 $\hat{R}$ value in R

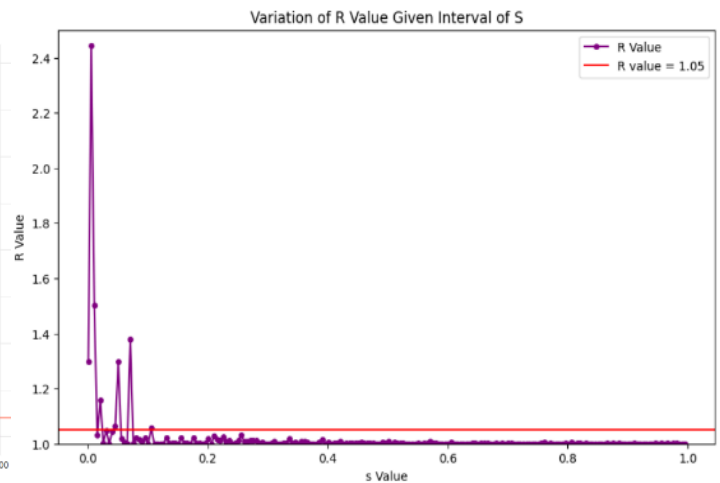


Figure 1.4 $\hat{R}$ value in Python

The graph above shows how the $\hat{R}$ vary depends on the s within 0.001 to 1. The red colour line shows the $\hat{R}$ value when it's equals to 1.05. It shows that mostly the samples have $\hat{R}$ less than 1.05. This means the samples has successfully converge to target distribution ($f(x)$). The closer $\hat{R}$ to 1, the better it is. In figure 1.3 & 1.4, the initial $\hat{R}$ starts from high value and fluctuates. It's because of the algorithm is not converged in the initial stage. However, it's improved quickly and remains stable which is under 1.05 $\hat{R}$ value, that shows it's a good estimate of targeted distribution. This mean as the s vary from 0.001 to 1 by 0.005. When s around 0.001, the $\hat{R}$ is fluctuated and above 1.05 that shows it might be too slow to converge. While the greater the s , the better the conversion, especially when s around 0.125 onwards. The goal is to find the best s (step size) so that the algorithm could be converge properly,

```
Sample_mean: -0.09436207
Standard Deviation: 1.481309
```

Figure 1.5 Sample Mean and Standard Deviation in R

```
Sample_mean: 0.07910639097804344
Standard Deviation: 1.3159301702149588
```

Figure 1.6 Sample Mean and Standard Deviation in Python

The sample mean is around -0.09 and 0.079 in R and Python respectively. While the standard deviation is about 1.48 and 1.32. This is also the estimates of the mean and standard deviation on its Monte Carlo.

2.0 Flights Analysis 2000 - 2004

In this report, delay is defined as all flights that arrive or depart exceed the scheduled time by 15 minutes or more (OAG). Due to the high correlation around 0.8945 between departure and arrival delay, there will be separated calculations for these. Departure delay is when the flight departs for 15 minutes or more after the schedule (DepDelay ≥ 15). While arrival delay is defined when the flight arrives 15 minutes or more and is not caused by departure delay (DepDelay < 15 & ArrDelay ≥ 15). This has been done to avoid double counting as departure delay mainly comes with arrival delay.

2.1.1 Problem Analysis

In order to answer what is the best day of week, average delay and ratio of delay could be calculated here. Average delay is calculated by sum of the delay divided by number of delayed flights. The ratio of delay will calculate the number of delayed flights in a day divided by total flight. Frequency analysis won't be used here, as low frequency doesn't necessarily mean low possibility of delay, but because of low number of flights.

To answer which time is the best, the data will be divided into 6 categories using CRSDepTime. Each category contains 4 hours. There will be "0-4" represents when the time is 0:00 to 4:00, "4-8" which is after 4:00 to 8:00 ($>4:00$), "8-12" which is after 8:00 to 12:00 ($>8:00$), etc.

2.1.2 Data Collection

The Harvard Dataverse provides the dataset that will be used here. There will be five years of data from 2000 to 2004. The data will be read from the csv. It will be automated using function in both R and Python.

2.1.3 Data Cleaning, Preprocessing, and Wrangling

The data from 2000 to 2004 will be combined into one data frame. Data is taken when the flight is not cancelled and not diverted. Instead of all columns, DayOfWeek, Year, Diverted, Cancelled, DepDelay, and ArrDelay will be selected in this analysis. Diverted and Cancelled must be selected so that filtering can be done. Since, there's high correlation between ArrDelay and DepDelay then the analysis will be separated.

There will be filtering of data to ensure whether it's classified as departure delay or arrival delay. Therefore, the missing value will be automatically removed. For example, filter the rows that have DepDelay ≥ 15 (departure delay) which removes the NA values on DepDelay. This applied in arrival delay, where ArrDelay ≥ 15 and DepDay < 15 , that automatically remove the NA values. In arrival delay analysis, there will be NA's in DepArr. Since its arrival is delayed, we won't calculate using DepArr. Year will be converted to date format.

While the average delay will be calculated using the given data, the ratio of delay will stored in a new column call Ratio_of_delay. First, when the DepDelay ≥ 15 , it will be denoted as 1, otherwise 0. Then it can be used to calculate how many flight is delayed based on definition above which subsequently being used for ratio.

2.1.4 Visualization

To answer which day is the best, line plots and bar charts will be displayed. While for the best time, boxplot and heatmap are shown to have better understanding.

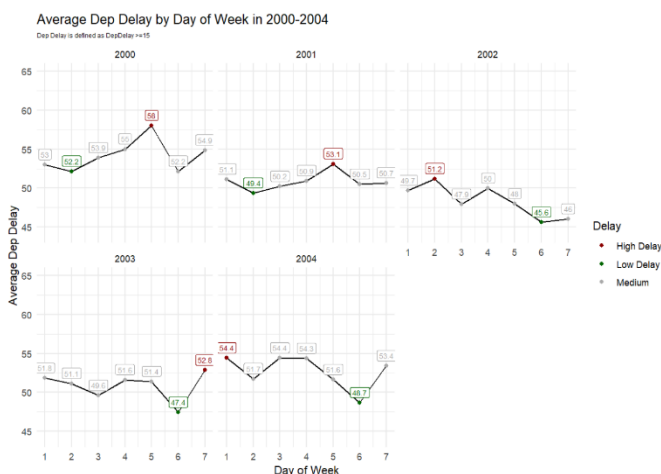


Figure 2.1 Average Departure Delay by Day of Week in 2000-2004 in R

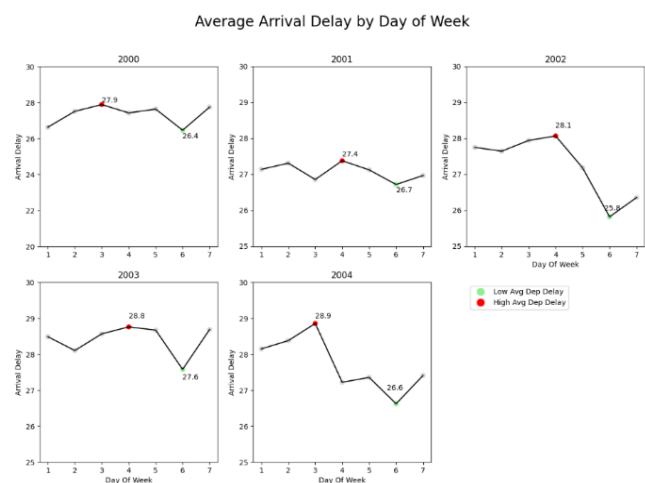


Figure 2.2 Average Arrival Delay by Day of Week in 2000-2004 in Python

Figure 2.1 & 2.2 shows the average departure and arrival delay by day of week in 2000-2004. The lowest delay is on Saturday, which is quite consistent across these 5 years, except in 2001 for the departure delay analysis. It was the lowest for Tuesday, at 49.4 minutes. However, the difference between Tues and Sat is less than 1 minute. In 2001 for departure delay, Tuesday have the lowest delay which is the same average departure delay with Saturday.

Overall, the difference between average departure delay is less than 6 minutes (small differences) over the years, which needs further analysis to determine which day is the best day to avoid delay.

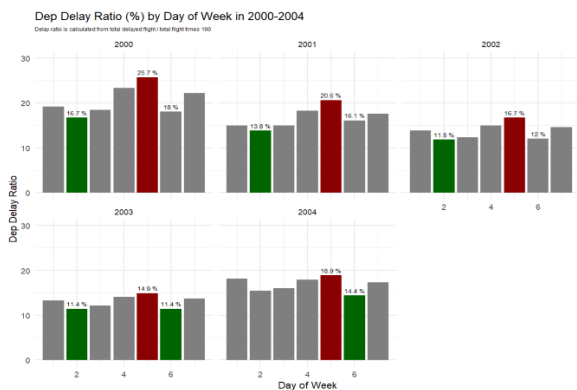


Figure 2.3 Ratio of Departure Delay (%) by Day of Week in 2000-2004 in R

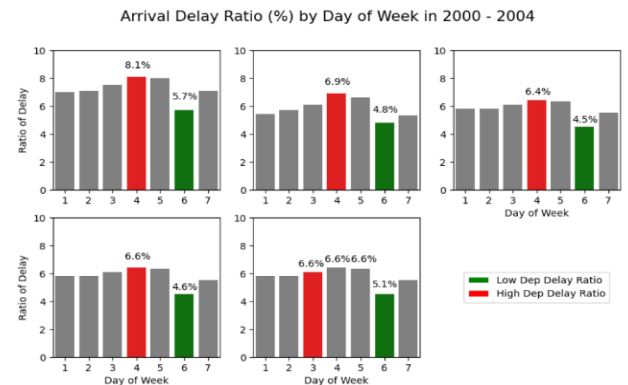


Figure 2.4 Ratio of Arrival Delay by Day of Week in 2000-2004 in Python

Frequency delay is not a good measure to determine how bad the delay is on a daily basis. As low frequency of delay could be a result from low number of flights, therefore ratio of flight delay must be calculated. Ratio is calculated based on the number of delayed flights divided by the total number of flights in a day. This will be displayed in percentage. Whether it's arrival or departure delay, there's a consistently high ratio on Friday. This means Friday has the highest risk of delay. From figure 2.3 & 2.4, Tuesday has the lowest ratio in 2000-2003 for departure delay. However, Saturday which is represented as "6" consistently has low ratio delay across the year. In overall, only 3 out of 10 graphs show that Sat is not the lowest ratio and it's possessed by Tuesday.

For arrival delay, Wed and Thursday are the worst. These days have the highest ratio of departure delay with high average delay. Although it's the highest, the difference between average and ratio delay in arrival delay is small. The ratio of arrival delay for all days are lower than 10% unlike departure delay with high variation. For departure delay, the highest ratio is in Friday across year. While in Sat, delay is unlikely to happen as there's a low ratio of delay. In 2000 the difference of ratio between Fri and Sat is around 7 percent. Due to high differences in the ratio of delay, this measure will be used to determine which day of the week is the best.

Tuesday might be good in 2000 to 2001, where the ratio and average delay is the lowest for departure delay. But in 2002 the trend is changed for Tuesday, as it's become the highest average delay, while the ratio of departure delays still the lowest. It means delay happens more frequent and the impact in minutes is less severe. But in the subsequent year, the ratio of delay for Tuesday is increasing, as well as average delay that is not the lowest anymore. Saturday is consistently having the lowest average ratio of delay as well as ratio of delay. Therefore, Saturday is concluded as the best day, with lower possibility of delay, with low average delay. While Friday must be avoided.

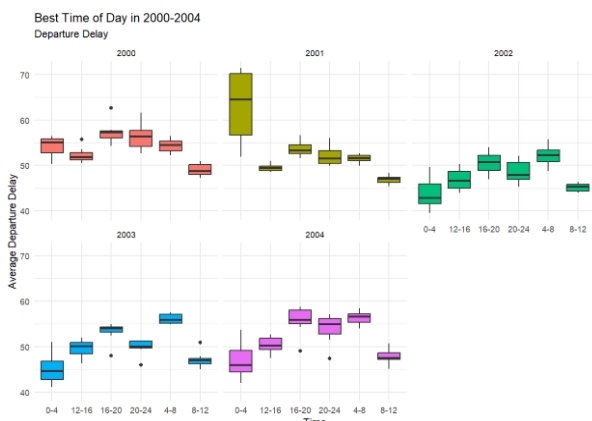


Figure 2.5 Average Departure Delay by Time Category in 2000-2004 in R

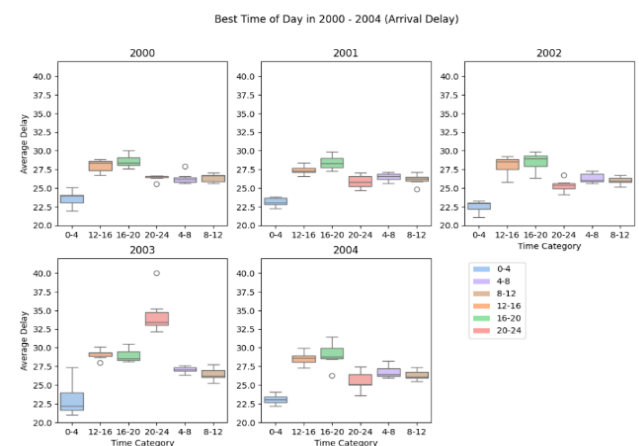


Figure 2.6 Average Arrival Delay by Time Category in 2000-2004 in Python

From figure 2.5 & 2.6, the average delay is mainly low at 0:00 to 4:00. The departure delay in this time category might be higher in 2000 and 2001 with left skewed distribution, however it's always the lowest figure in the other years including the arrival delay. It also has high variation in that time category. When it's at 4:00 to 8:00 the delays might not be the lowest for all 5 years, but it's consistently low with small variation. When it's 16:00 to 20:00 the average delay seems to be high. Even it's not always to be the highest, it's consistently higher than other time category across five years in both arrival and departure delay.

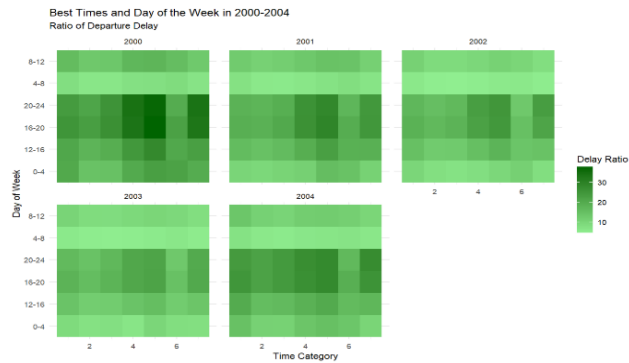


Figure 2. 7 Departure Delay by Time Category & Day of Week in R

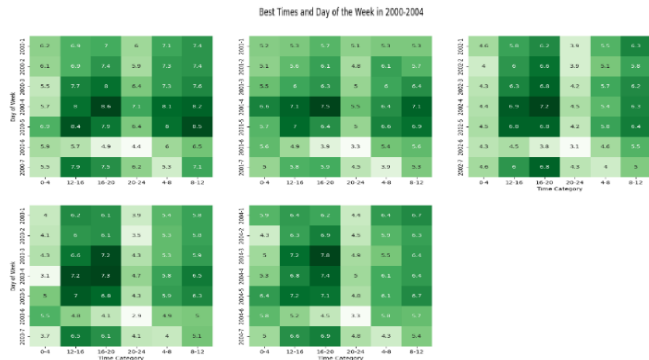


Figure 2. 8 Arrival Delay by Time Category & Day of Week in Python

Heatmap is displayed to see the ratio of delay across the weeks, years, and time category. The differences in the ratio of arrival delay are minor with less than 2% and all figures are under 10% which is unlikely useful for analysis. Therefore, departure delay will be seen more. Departure delay ratio is the lowest at 4-8 and it's consistent across 5 years. While in 16-20 and 20-24 the ratio delay is much higher which is shown in dark colour. The ratio of departure delay is within 5% to 30%. This shows high differences in ratio between each time category. Therefore, ratio measures will be prioritised to determine the best time to avoid delay. For arrival delay, 12-16 and 16-20 have the higher ratio of arrival delay than other time. 16-20 also have high average of delay. As well as high ratio of departure and arrival delay.

To conclude, instead of 0-4 that have lower average delay in minutes, we choose 4-8. Even though 4-8 have a higher wait of delay in minutes, it has a much lower possibility of delay to happen. Therefore, 4-8 is the best time to fly and minimise delay. 16:00 to 20:00 must be avoided as it has a high ratio of delay and high average ratio of delay. From the heatmap, Saturday is the best day to fly across all time categories which reinforces the previous argument.

2.2.1 Problem Analysis

To answer whether old planes suffer more delay, plane-data.csv will be used. The years chosen are from 2000 to 2004. Delay will be calculated based on arrival and departure delay, which is the same method with the previous question.

2.2.2 Data Cleaning, Preprocessing, and Wrangling

The data from 2000 to 2004 will be combined into one data frame. Then left join will be performed to merge the data with plane-data.csv based on TailNum. As there's differences in column names in this data, we will replace the tailnum into TailNum in order to perform left join. The delay analysis will be the same as the previous question. As there's a high number of NA, the data couldn't be removed. However, when it goes to visualisation, there will be adjustment.

Subset of data will be created with certain columns. The data will filter out the cancelled flight and diverted flight. The Year column which represents the day of flight will be compared with the "year" column that represents the year of manufacturer. When the "year" is "None" and 0 it will be converted to NA. In certain case, where Year < year, the rows will be removed. As it's impossible to have flight before the plane is manufactured. Age column will be created to calculate the difference between Year and year. The age category also has been created, which is "0-10" represents the age of plane from 0 to 10 years, "11-20" represents 11-20 years old, etc.

2.2.3 Visualisation

The scatter plot shows the relationship between age and average delay with linear regression. Here NA will be removed. Whether it's departure or arrival delay, it shows a weak linear relationship between age of plane and delay. There's almost no relationship between them and delay is not explained by age. However, this conclusion must be seen with cautious as there's many NA's values that are not included.

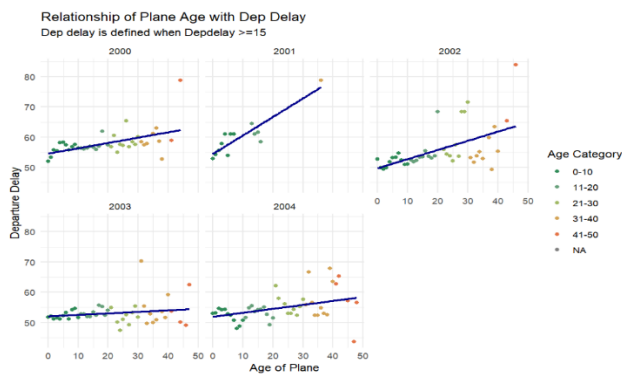


Figure 3. 1 Relationship of Plane Age with Departure Delay in R

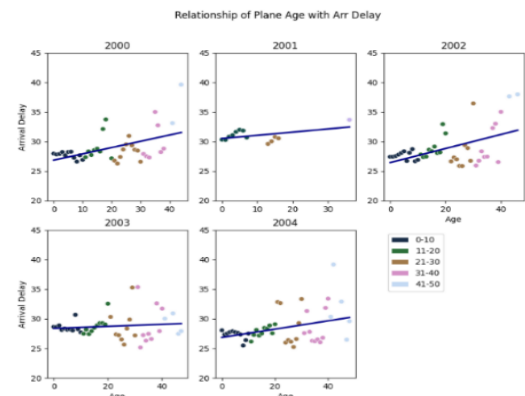


Figure 3. 2 Relationship of Plane Age with Average Delay in Python

Figure 3.1 & 3.2 show the relationship of age of plane and delay. There're huge NA's values in this analysis, especially in 2001. The relationship seems to be weak and mostly is almost no relationship.

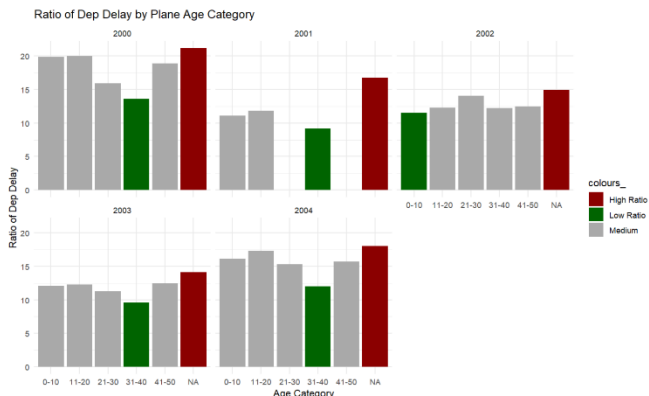


Figure 3. 3 Ratio of Departure Delay by Plane Age Category in R



Figure 3. 4 Ratio of Arrival Delay by Plane Age Category in Python

In figure 3.3 & 3.4, it shows the ratio of delay with the age category. In the departure delay analysis, it has a low ratio mainly in 31-40 years old. However, the NA's values are the highest, so that analysis must be seen with care. As the arrival delay ratio of NA's value is the lowest across the year, then ignoring NA's might not affect the analysis. The arrival delay indicates that older plane is not suffering delay more than younger plane. It's the highest proportion of delay when the age is between 0 to 10. Since there's huge NA's values, it might influence the result of analysis. So that, the conclusion must be seen with cautious. Whether it's average delay or ratio of delay, it's not explaining the age of the plane. The older the plane, it doesn't necessarily have higher delay. There's no sufficient evidence to say older plane suffers more delays.

```
[1] 0.009022464
```

```
[1] -0.0003733672
```

Furthermore, the correlation of departure and arrival delay with plane age respectively seems to be low and almost no linear relationship between them. Even the arrival delay is slightly

negative which against the statement that old plane suffers more. The fact that correlation under 0.3 will be seen as no relationship, which reinforces previous conclusion.

Older planes might not suffer more delays since it has ongoing maintenance and regular check. In addition, age is not the only factor to cause delay. There's weather, air traffic issue, or other technical things that can cause delay more frequently.

3.1.1 Problem Analysis

The main objective is to fit logistic regression model and get the coefficient of features. In order to get it, model must be trained with selected features. Through the train and test data, the model will predict whether the flight is diverted or not. The accuracy, specificity, and sensitivity can be calculated later. The target variable is Diverted which is in binary form, 0 for not diverted and 1 for diverted flight. The model will be performed in both R and Python, however the report will shows the rmd result.

3.1.2 Data Cleaning, Preprocessing, and Wrangling

The data from 2000 to 2004 will be combined into one data frame. The data will be filtered and choose the data that is not cancelled (Cancelled == 0). DepDelay, UniqueCarrier, Diverted, CRSDepTime, Distance, DayOfMonth, DayOfWeek will be selected to perform analysis. The process will be done per year instead of 5 years directly, due to the large dataset. The "Diverted" column will be converted to factor (R) with 2 levels or category (Python). The CRSDepTime and CRSArrTime will be treated as continuous. DayOfWeek and DayOfMonth will be converted into factor since the

correlation of “Spearman” method is higher that indicates potential monotonic relationship. For example, the increase of day of month might not increase the probability of diverted, but it could be in certain day due to seasonality, that highly influence diverted flight.

The reason why other features like CarrierDelay, CRSElapsedTime, etc are not included because of the huge NA’s values. While NA’s values in ArrTime and DepTime may not that much, but it has high correlation with CRSArrTime and CRSDepTime respectively that introduce multicollinearity. CRSElapsedTime seems to be useful, it have 0.99 correlation with Distance. Here distance is chosen since it has better correlation with target variable. Certain UniqueCarrier is useful for analysis that’s shown on correlation analysis (python). As well as DayOfWeek and DayofMonth that have correlation with Diverted (Spearman method in Python).

The data will be separated into train and test data. The train data will contain 50% of the original data. The train data is used to train the model, while test data is unseen data that will be used to see the performance of model. There's large imbalance train data between 0 and 1 on targeted variable (Diverted). It will use mlr learner to perform model.

3.1.3 Visualisation

The figure shows the coefficient of features after training the model.

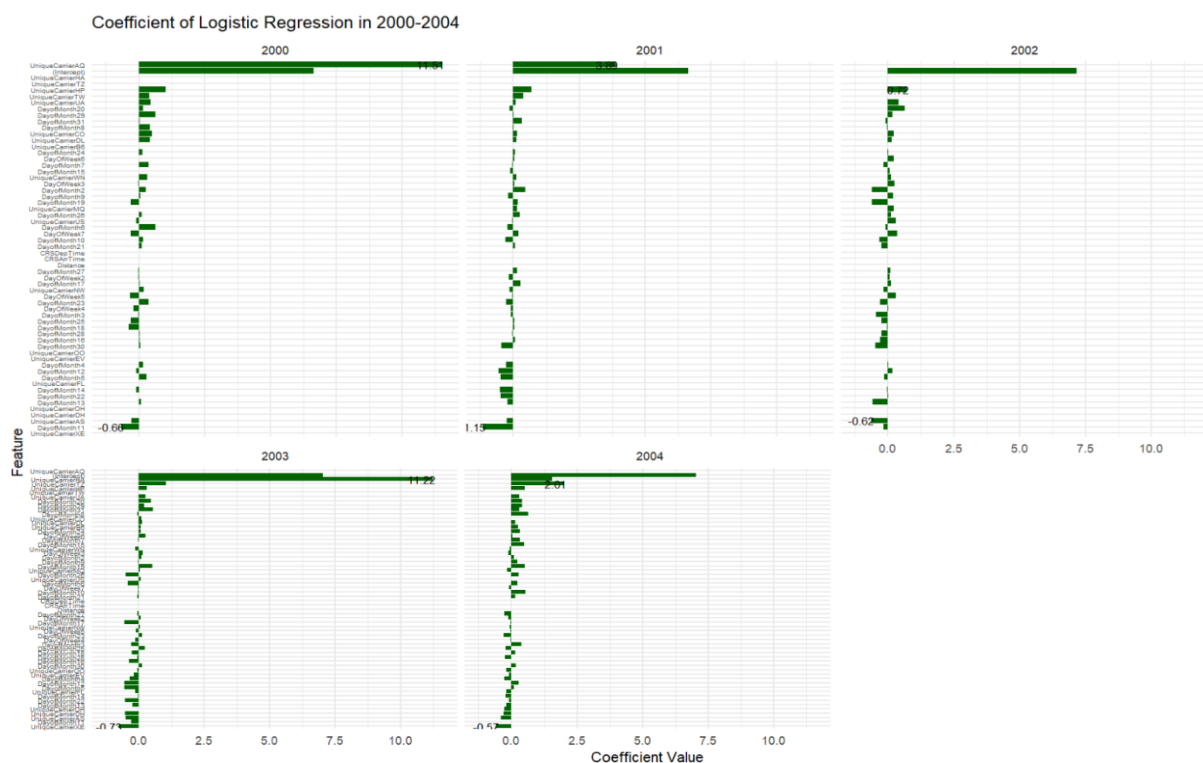


Figure 4. 1 Coefficient of Logistic Regression in R

The coefficient of logistic regression will be used to see how the flight diverted will be influenced when any selected features change in one unit. When it's positive coefficient, the increase in one unit of it will make diverted happens more likely. When it's negative coefficient, increase in one unit of selected feature will result in lower possibility of diverted. Therefore, positive coefficient show positive relationship and negative coefficient indicates negative relationship. Figure 4.1 display the coefficient that might be useful to predict diversion. While not all features are available in all five years, such as UniqueCarrierAQ. It possesses the large coefficient (positive) in 2000 and 2001, but not available in the other years. As well as UniqueCarrierXE that has negative coefficient in 2003 and 2004 but not available in 2000 -2003. Then for further analysis, features with high coefficient will be chosen, given it's available in all 5 years.

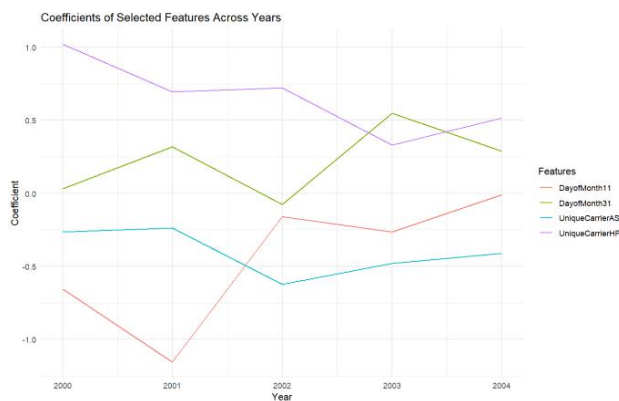


Figure 4.2 Coefficient of Logistic Regression by Year in R

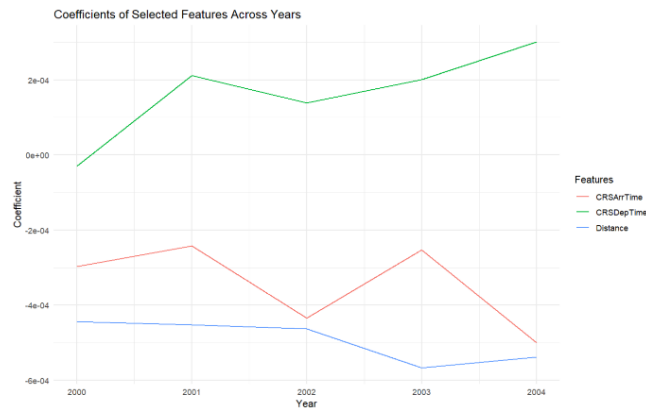


Figure 4.3 Coefficient of Logistic Regression by Year in R

The DayOfMonth11 and 31 have negative and positive coefficient respectively. In 2000 - 2001, the coefficient is quite strong for DayOfMonth11. It reached more than negative one coefficient in 2001. While in 2002-2004 the coefficient might be a bit smaller than previous years. This means when day of month is 11, the flight will unlikely to be diverted since it's negative coefficient. However, the level of "unlikely" will be determined on small or high coefficient. The lowest the coefficient means the stronger influence for negative coefficient. Then, when the day of month is 11 in 2001, the flight has lower odds ratio of diversion.

DayOfMonth31 have positive coefficient. Similar like DayOfMonth11, it has higher coefficient in 2000 - 2001, but slightly decreases in 2001-2002. But here, it's increasing again during 2003-2004 period. It means the DayOfMonth31 is likely to make diversion happens, especially in 2003. Therefore, certain day might have higher odds of diversion.

UniqueCarrierAS have negative coefficient which is high in 2001-2002, that indicates when the unique carrier is AS, the diverted flight is unlikely to happen. While UniqueCarrierHP reach the highest positive coefficient in 2000 and decreasing in subsequent years. It suggests when carrier is HP the flight is likely to be diverted. This indicates certain carrier or airline will have higher potential of diversion flight.

Figure 4.3 displays the coefficient of CRSDepTime, CRSArrTime, and Distance. Even though it's small, it's measured in one unit. Here CRSDepTime is in the range of 0 to 2400 as well as CRS ArrTime. While distance is per miles. Then it will be highly impactful features when the unit increase is not one but more. CRSArrTime and CRSDepTime have negative coefficient and reach it's highest at 2004 and 2003 respectively. It means when these features increased by one minute, diverted is unlikely to happen. While distance have positive coefficient and reach its peak in 2004, then the increase in one mile will make diversion more likely to happen. In the flight case, distance won't be just increase in 1 mile, it will be more additional unit that mean the coefficient will be impactful. Then longer flight will have higher chance to be diverted.

In addition, when year 2000, the model has error for 0.00259 which is very small. While there's high imbalance of data, then the performance of model might be affected. The threshold will be adjusted so that it will have slightly better result as there's high imbalance data. After adjustment the sensitivity is around 14.13%, while the specificity is 93.7%. This suggests the model true positive in low, which is crucial in our case. The AUC is around 0.64 which means it's slightly better than random guess. If value less than 0.5 it means the model is not even better than random guessing. It applied the same in the rest of the years.

This coefficient and model are highly affected by imbalance data. The data here is highly imbalance indicates there's room for improvement. SMOTE could be performed to balance the data, whether it's under sampling or over sampling. This has been done only in train data, not in test data. Other method such as glm() on R can be used, so that p-value can be detected when running each model. After getting the summary of the model, feature with significant p-value will be removed (>0.05) because it's not rejecting the hypothesis test where the features don't have any impact on diverted. After that, it could rerun the model by filtering specific feature and see the improvement of the model. Re-running and adjusting the model in multiple times might help to improve model, but it shouldn't be too much, as there's risk of overfitting. Other alternatives such as ridge and lasso can be used to improve the model since penalty could be applied. The reason why it's not being used glm() to run the model at the first analysis, is too large dataset and mlr learner is more efficient.

References

OAG. (2023). Retrieved from OAG: <https://www.oag.com/blog/defining-late-on-time-performance-in-the-aviation-industry#:~:text=What%20Does%20Good%20On%2DTime,the%20cost%20of%20achieving%20them.>